Intent-Aware Deception: Real-Time Attacker Profiling via Hybrid Rule–ML Classification of Honeypot Command Streams

Rakshita Shetty
Department of Computer Science and Engineering
Intent-Aware Deception System — Research Project

***Abstract*—Traditional honeypots excel at capturing attacker activity but stop short of interpreting it: they log commands without inferring the adversary's underlying goal. We present an intent-aware deception system that transforms raw honeypot telemetry into real-time, explainable attacker profiles. Captured shell commands from an isolated medium-interaction honeypot are streamed through an asynchronous pipeline that normalises each command, extracts lexical features, and classifies it into one of ten MITRE ATT&CK-aligned intent classes using a hybrid classifier: a deterministic rule engine, a calibrated machine-learning core, and an optional large-language-model semantic layer, arbitrated by explicit confidence precedence. Per-session and per-attacker profiles are assembled into multi-stage attack trajectories, and each session receives an explainable risk score (0-100). The ML core - a calibrated linear support-vector machine over a combined word- and character-n-gram TF-IDF feature space - achieves 94.49% accuracy and 0.9229 macro-F1 on a leakage-controlled held-out set of 272 commands; a stricter leave-families-out protocol yields 67.87% accuracy, reported explicitly to characterise generalisation honestly. The full system runs end-to-end in real time and is reproducible from a fixed seed. Fusing deterministic rules, calibrated ML, and an optional LLM yields profiling that is simultaneously accurate, real-time, and auditable - a combination none of the three achieves alone.**

***Index Terms*—honeypot, cyber deception, attacker profiling, intent classification, MITRE ATT&CK, machine learning, text classification, threat intelligence, explainable security, support vector machine.**

I. INTRODUCTION

Honeypots - decoy systems with no legitimate purpose - are a cornerstone of cyber deception: because no benign user should ever touch them, every interaction is, by definition, suspicious [1], [2]. Medium-interaction honeypots such as Cowrie [3] emulate a Unix shell, enticing attackers to reveal their tooling and tradecraft while safely containing them. However, the dominant output of such systems is a log: a chronological record of commands that a human analyst must still read and interpret. At scale, this interpretation burden is the bottleneck.

The gap we address is semantic: existing honeypots tell you what an attacker typed, but not why. Knowing that an attacker ran `cat /etc/shadow` followed by `wget http://.../x.sh` and then `rm -rf /var/log/auth.log` is far more actionable when the system recognises this as credential access -> execution -> defensive cleanup - a coherent attack progression - and raises a scored alert automatically.

This paper presents an intent-aware deception system that closes that gap in real time. Our contributions are:

1) A 10-class, MITRE ATT&CK-aligned intent taxonomy for shell commands, with a deterministic mapping from each intent to an ATT&CK tactic/technique and to a kill-chain attack stage (Section IV).
2) A hybrid classifier that arbitrates a deterministic rule engine, a calibrated ML core, and an optional LLM semantic layer by explicit confidence precedence, recording the deciding source for every event so that every classification is auditable (Section VI).
3) An explainable additive risk-scoring engine (0-100) that combines per-intent weights with behavioural boosters and maps to four severity tiers with automatic alerting (Section VII).
4) A complete, real-time, reproducible implementation - asynchronous capture pipeline, MongoDB persistence, FastAPI REST/WebSocket API, and a live React analyst dashboard - with the honeypot strictly isolated so no captured command is ever executed on the host or backend (Sections IX, XII).
5) An honest experimental evaluation with leakage control and a stricter leave-families-out protocol, reporting both the favourable and the conservative numbers (Sections X-XI).

II. RELATED WORK

Honeypots and deception. Spitzner [1] and Provos and Holz [2] established honeypots as instruments for observing attacker behaviour; Cowrie [3] is the de-facto medium-interaction SSH/Telnet honeypot, emulating a shell and logging structured JSON events. These systems are excellent sensors but are typically paired with manual or signature-only analysis.

Command and intrusion classification. Text-classification techniques over command lines and system logs are well established; TF-IDF vector-space representations [4] with linear classifiers such as support-vector machines [5] remain strong, interpretable baselines for short, high-dimensional sparse text. Character-n-gram features are known to be robust to the lexical variation and obfuscation typical of malicious inputs.

Adversary modelling. MITRE ATT&CK [6] provides a standard taxonomy of adversary tactics and techniques and

has become the lingua franca for describing intrusions. Mapping observed behaviour onto ATT&CK tactics enables shared vocabulary and downstream automation.

LLMs for security triage. Recent work applies large language models to security-log summarisation and triage. LLMs offer strong semantic generalisation but introduce latency, cost, non-determinism, and availability dependencies - properties ill-suited to a mandatory real-time path.

Our position. Rather than choosing one paradigm, we fuse them: deterministic rules give precision and auditability on known signatures; a calibrated ML model gives generalisation and probabilistic confidence; and an LLM is an optional fallback for genuinely novel or ambiguous commands. The novelty is not any single classifier but the arbitration architecture and its integration into a real-time, explainable, end-to-end profiling system.

III. SYSTEM ARCHITECTURE

The system is a streaming pipeline from honeypot capture to analyst dashboard (Fig. 1).

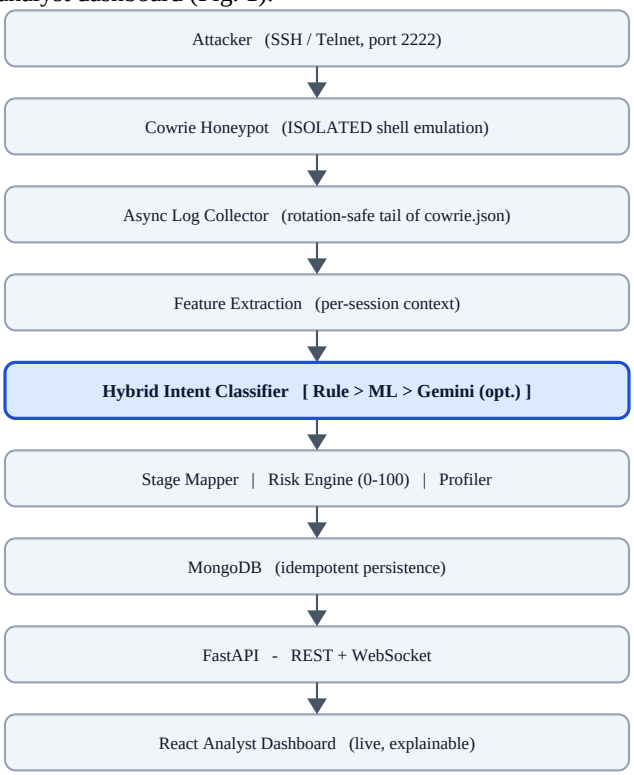

Fig. 1. End-to-end streaming architecture: honeypot capture through hybrid classification, risk scoring, persistence, and the live analyst dashboard.

Data flow. (1) The honeypot captures a login/command and appends a JSON event to its log. (2) An asynchronous collector tails the log and emits normalised events without blocking. (3) Feature extraction attaches session context (command counts, login failures, timing). (4) The hybrid classifier assigns an intent, a deciding source, and a confidence. (5) The attack-stage mapper places the intent on the kill chain; the risk engine scores the session; the profiler aggregates per-session and per-attacker state. (6) Everything is persisted to MongoDB and (7) exposed via a FastAPI REST + WebSocket API to (8) a live React dashboard.

Design principles. (i) ML is mandatory, the LLM is optional - the system starts and operates fully without any API key. (ii) Every decision is auditable - each event stores its deciding source and evidence. (iii) Strict isolation - the honeypot is the only component that touches attacker input, and it only emulates a shell.

IV. INTENT TAXONOMY AND MITRE MAPPING

We define ten intent classes, each mapped deterministically to a MITRE ATT&CK tactic/technique and to a kill-chain attack stage. The mapping is a static table in code (`taxonomy.py`), so intent -> tactic -> stage is reproducible, not inferred.

TABLE I
INTENT TAXONOMY, MITRE ATT&CK; TACTIC, AND ATTACK STAGE

Intent class	MITRE tactic	Representative technique(s)	Stage
RECONNAIS-SANCE	Reconnaissance (TA0043)	Active Scanning (T1595); Network Service Discovery (T1046)	RECON
DISCOVERY	Discovery (TA0007)	System Information (T1082); File/Dir Discovery (T1083); Process Discovery (T1057)	DISCOVERY
CREDENTIAL_ ACCESS	Credential Access (TA0006)	OS Credential Dumping (T1003); Unsecured Credentials (T1552)	CRED_ ACCESS
EXECUTION	Execution (TA0002)	Command & Scripting Interpreter (T1059)	EXECUTION
PRIVILEGE_ ESCALATION	Privilege Escalation (TA0004)	Abuse Elevation Control (T1548); Exploitation for Priv-Esc (T1068)	PRIV_ESC
PERSISTENCE	Persistence (TA0003)	Account Manipulation (T1098); Scheduled Task (T1053); Create Account (T1136)	PERSIST-ENCE
EXPLOITATION	Execution / Initial Access	Exploit Public-Facing Application (T1190)	EXECUTION
EXFILTRATION	Exfiltration (TA0010)	Exfil over C2 (T1041); Exfil over Alt. Protocol (T1048)	EXFIL
CLEANUP	Defense Evasion (TA0005)	Indicator Removal - clear logs/history (T1070)	CLEANUP
OTHER	-	(no confident tactic)	UNKNOWN

Two deliberate choices: EXPLOITATION maps to the EXECUTION stage (an exploit is a means of code execution), and OTHER maps to UNKNOWN (we do not force a kill-chain position onto unclassified input).

V. MACHINE-LEARNING METHODOLOGY

A. Task

Supervised multi-class classification of a single shell-command string into one of the ten intent classes. The classifier consumes only the raw command text; behavioural session features are reserved for the risk engine (Section VII).

B. Dataset

The corpus is generated programmatically from 242 curated command template families. Each family declares an intent and a parameterised template; templates are expanded by substituting from value pools (IPs, ports, paths, payload names, CVE IDs), capped at 14 variants per family, then de-duplicated. After removing 5 exact duplicates the final

corpus is 1360 unique commands across 10 classes (Table II). Labels are assigned by construction, and the `family_id` is retained as a group key for the robustness split.

TABLE II
CLASS DISTRIBUTION (SAMPLES / FAMILIES)

Intent	Samples	Families
RECONNAISSANCE	80	28
DISCOVERY	134	35
CREDENTIAL_ACCESS	121	27
EXECUTION	318	24
PRIVILEGE_ESCALATION	57	30
PERSISTENCE	162	20
EXPLOITATION	100	18
EXFILTRATION	184	18
CLEANUP	87	18
OTHER	117	24
Total	**1360**	**242**

Fig. 2. Class distribution of the 1360-command corpus across the ten MITRE-aligned intent classes.

C. Features

A scikit-learn `FeatureUnion` of two TF-IDF representations: a word vectoriser (analyzer=word, whitespace tokenisation, n-grams (1,2), sublinear_tf) and a character vectoriser (analyzer=char_wb, n-grams (3,6), sublinear_tf). The whitespace tokeniser keeps flags, paths, and URLs intact (`-O`, `/etc/shadow`), and the character n-grams capture sub-token cues (flag fragments, path fragments, obfuscation) that a word-only model misses - the single most important design choice for the shell-command domain.

D. Model selection

Three candidates over the identical feature space were compared by 4-fold stratified cross-validation on the training split only, scoring macro-F1 (Table III). The winner is a calibrated linear SVM: `LinearSVC(C=1.0, class_weight=balanced)` wrapped in `CalibratedClassifierCV(method=sigmoid, cv=3)`. Calibration (Platt scaling [7]) converts SVM margins into probabilities that the runtime hybrid layer thresholds on.

TABLE III
MODEL SELECTION (4-FOLD CV MACRO-F1, TRAINING SPLIT)

Candidate	Estimator	CV macro-F1
linsvc_calibrated (selected)	Calibrated LinearSVC	0.8539 +/- 0.018
logreg	LogisticRegression (C=3.0)	0.8482 +/- 0.017
complement_nb	ComplementNB (alpha=0.3)	0.8146 +/- 0.028

E. Protocol and leakage control

A stratified 80/20 split (1088 train / 272 test, random_state=1337). The code asserts that no identical command string appears in both train and test - this leakage control is why the headline accuracy is a realistic 94%, not an inflated 100%. The shipped artefact is refit on all 1360 samples and serialised with metadata; loading is lazy and degrades gracefully to rules+LLM if the artefact is absent.

VI. HYBRID CLASSIFICATION

At inference, three sources are arbitrated by explicit confidence precedence (`hybrid_classifier.py`):

1) **Strong RULE:** rule confidence >= 0.90 and label != OTHER -> source RULE.
2) **Confident ML:** calibrated ML probability >= 0.40 -> source ML (HYBRID if the rule agrees).
3) **Gemini (opt.):** consulted only when 1 and 2 are inconclusive -> source GEMINI.
4) **Best-available:** highest-confidence non-OTHER among rule/ML.
5) **Fallback:** nothing confident (>= 0.10) -> OTHER, source FALLBACK.

- The rule engine comprises 13 rules over 43 curated regular-expression patterns, each with an attached confidence; a strong (>=0.90) non-OTHER match is authoritative - deterministic, instant, and auditable.
- The ML core decides when no strong rule fires but its calibrated probability is >= 0.40; when the rule agrees with the ML label the event is recorded as HYBRID, otherwise ML.
- Gemini (`gemini-2.5-flash`) is consulted only when neither rule nor ML is decisive. Its output is gated (accepted at >=0.85) and its input sanitised. It is never on the hot path for known attacks, and the system runs fully without it.
- Every event stores its deciding source (RULE/ML/HYBRID/GEMINI/FALLBACK) and, for ML/HYBRID, the calibrated confidence - surfaced on the dashboard so no decision is a black box.

This arbitration is the core systems contribution: it combines the precision and auditability of rules, the generalisation and calibrated confidence of ML, and the semantic reach of an LLM, while keeping ML mandatory and the LLM optional.

VII. EXPLAINABLE RISK SCORING

Each session receives an integer risk score in [0, 100] from an additive engine (`risk_engine.py`): a base weight for the highest-severity observed intent, plus behavioural boosters derived from session context, clamped to 100.

TABLE IV
BASE INTENT WEIGHTS

Intent	Weight
OTHER	0
RECONNAISSANCE	10
DISCOVERY	15
CLEANUP	15
EXECUTION	20
CREDENTIAL_ACCESS	25
PRIVILEGE_ESCALATION	30
PERSISTENCE	30
EXPLOITATION	30
EXFILTRATION	35

TABLE V
BEHAVIOURAL BOOSTERS

Condition	Points
Any failed authentication	+5
Brute-force pattern (>= 3 failed logins)	+10
Rapid command bursts (high command rate)	+5
Deep session (>= 5 commands)	+5
High intent variety (>= 4 distinct intents)	+5

The score maps to four severity tiers - LOW 0-24, MEDIUM 25-49, HIGH 50-74, CRITICAL 75-100 - and a session reaching >= 75 raises an automatic alert. Because the formula is additive and transparent, every score is fully explainable.

VIII. ATTACKER PROFILING

The profiler maintains two aggregation levels. A session profile records the intent sequence, dominant intent, furthest attack stage, command/login counts, duration, risk score, and severity - assembling isolated events into a multi-stage trajectory (e.g. reconnaissance -> discovery -> credential-access -> privilege-escalation -> exfiltration -> cleanup). An attacker profile aggregates all sessions from one source IP: total sessions/events, peak and average risk, dominant intent, and first/last-seen timestamps - answering whether a host is a one-off scanner or a persistent, escalating threat.

IX. IMPLEMENTATION

Backend. Python 3.13, FastAPI, Motor (async MongoDB). The capture->classify->score->persist path is fully asynchronous so ingestion never blocks. The REST/WebSocket surface includes health, dashboard statistics, events, sessions (with on-demand MITRE mapping and optional Gemini analysis), attackers, alerts, and six analytics endpoints, plus `/ws/events` and `/ws/alerts` for live streaming.

Frontend. A React 19 + Vite + Tailwind + Recharts single-page analyst dashboard with five sections - Overview, Live Sessions, Attacker Profiles, Threat Alerts, Analytics. Each event row shows an intent badge, severity, and a classification-source badge (with ML confidence), realising the explainable goal on screen.

Deployment and reproducibility. The full stack - MongoDB, Cowrie, API, and dashboard - is containerised via Docker Compose and starts with a single command. A fixed RANDOM_SEED = 1337 and pinned scientific-stack versions (scikit-learn 1.9.0, numpy 2.5.2) make training and evaluation deterministic and reproducible.

X. EXPERIMENTAL EVALUATION

A. Primary evaluation (leakage-controlled held-out set)

On the 272-command held-out test set the calibrated linear SVM achieves the metrics in Table VI, with per-class results in Table VII. The confusion matrix (Fig. 3) and per-class F1 scores (Fig. 4) visualise where the model is strong and where intents overlap.

TABLE VI
HELD-OUT PERFORMANCE (272 TEST COMMANDS)

Metric	Value
Accuracy	94.49%
Macro Precision	0.9367
Macro Recall	0.9158
Macro F1	0.9229
Weighted F1	0.9436

TABLE VII
PER-CLASS RESULTS (HELD-OUT)

Intent	Prec.	Recall	F1	Supp.
CLEANUP	1.000	0.941	0.970	17
CREDENTIAL_ACCESS	0.828	1.000	0.906	24
DISCOVERY	0.828	0.889	0.857	27
EXECUTION	1.000	1.000	1.000	64
EXFILTRATION	1.000	1.000	1.000	37
EXPLOITATION	1.000	1.000	1.000	20
OTHER	0.909	0.870	0.889	23
PERSISTENCE	0.970	1.000	0.985	32
PRIVILEGE_ESCALATION	1.000	0.833	0.909	12
RECONNAISSANCE	0.833	0.625	0.714	16

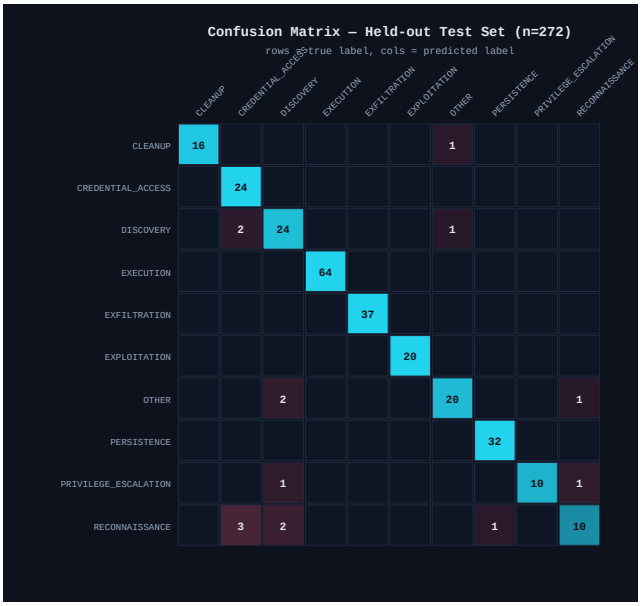

Fig. 3. Held-out confusion matrix (272 commands). Errors stay within neighbouring intents; none crosses into an unrelated tactic.

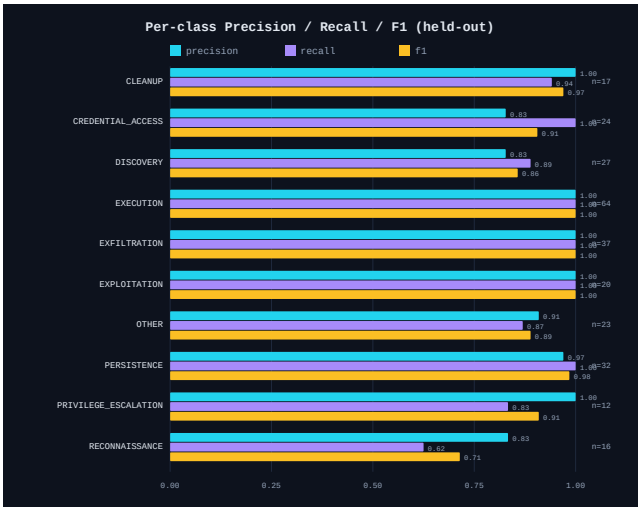

Fig. 4. Per-class F1 on the held-out set. EXECUTION, EXFILTRATION, and EXPLOITATION reach perfect F1; RECONNAISSANCE is weakest.

B. Robustness evaluation (leave-families-out)

To test generalisation to unseen command templates, we hold out whole families so that no test template appears in training (Table VIII, Fig. 5). This is a much harder test and the score is lower by design.

TABLE VIII
PRIMARY VS. ROBUSTNESS EVALUATION

Protocol	Acc.	Macro-F1	Train/Test
Primary (stratified, leakage-ctl.)	94.49%	0.9229	1088/272
Robustness (leave-families-out)	67.87%	0.5267	1055/305

Fig. 5. Primary vs. leave-families-out accuracy and macro-F1. The gap quantifies the lexical model's generalisation limit and motivates the hybrid design.

C. End-to-end live verification

Beyond offline metrics, an end-to-end script drives all ten intents through the running system and a verifier confirms each captured command carries a classification source and, for ML/HYBRID, a real calibrated confidence - demonstrating the ML core is genuinely in the live pipeline, not merely evaluated offline.

XI. RESULTS AND DISCUSSION

The model classifies attacker intent with 94.49% held-out accuracy and a balanced 0.9229 macro-F1, confirming that lexical features capture intent well for the majority of commands. Three classes - EXECUTION, EXFILTRATION, EXPLOITATION - reach perfect held-out F1, reflecting their highly distinctive vocabulary.

The weakest class is RECONNAISSANCE (F1 0.714): short probes such as `whoami`, `id`, and `uname` overlap lexically with DISCOVERY and CREDENTIAL_ACCESS, and it is the smallest test class (support 16). Crucially, the confusion matrix shows errors stay within neighbouring intents - no misclassification crosses into an unrelated tactic - so mistakes rarely change the kill-chain interpretation. This ambiguous boundary is exactly what the rule and optional-LLM layers exist to disambiguate at runtime.

The leave-families-out result (67.87%) is the honest headline for novel templates. A purely lexical model cannot fully generalise to command shapes it has never seen; we report this openly rather than quoting only the favourable split. It directly motivates the hybrid design: deterministic rules cover known-signature novelty, and the optional semantic LLM layer handles genuinely new commands - so the system's real-world robustness exceeds that of the ML core in isolation.

Finally, the architecture delivers explainability end-to-end: every event exposes its deciding source and evidence, and every risk score decomposes into transparent additive terms. An analyst can always answer 'why was this flagged?'

XII. LIMITATIONS AND ETHICAL / SAFETY CONSIDERATIONS

Limitations. (1) The training corpus, though realistic and family-diverse, is generated from curated templates rather than harvested from a live production honeypot; the leave-families-out result quantifies the resulting generalisation gap. (2) The classifier operates per command; richer sequence models could exploit inter-command context further. (3) Small classes (RECONNAISSANCE, PRIVILEGE_ESCALATION) have modest support.

Safety and ethics. The honeypot is strictly isolated and only emulates a shell - no captured attacker command is ever executed on the host or backend. All demonstration attacks use RFC-5737 documentation IP ranges (192.0.2.0/24, 198.51.100.0/24) that route nowhere, and no real malicious payloads are run. Secrets are confined to an untracked `.env`. The system is a defensive research tool and should only be deployed against infrastructure one owns.

XIII. CONCLUSION AND FUTURE WORK

We presented an intent-aware deception system that converts raw honeypot command streams into real-time, explainable attacker profiles. By fusing a deterministic rule engine, a calibrated ML core (94.49% held-out accuracy, 0.9229 macro-F1), and an optional LLM semantic layer under explicit confidence arbitration - and by scoring each session with a transparent additive risk engine - the system infers why an attacker acts, assembles multi-stage trajectories, and raises scored alerts, all while keeping ML mandatory, the LLM optional, and every decision auditable. We deliberately report a stricter leave-families-out score (67.87%) to

characterise generalisation honestly.

Future work. (1) Harvest and incorporate live honeypot traffic to close the template-generalisation gap. (2) Add sequence-aware modelling over the command stream. (3) Expand the rule/pattern library and intent taxonomy. (4) Study adversarial robustness to command obfuscation. (5) Explore automated adaptive deception - having the honeypot respond differently based on inferred intent.

REFERENCES

[1] L. Spitzner, Honeypots: Tracking Hackers. Boston, MA, USA: Addison-Wesley, 2002.

[2] N. Provos and T. Holz, Virtual Honeypots: From Botnet Tracking to Intrusion Detection. Upper Saddle River, NJ, USA: Addison-Wesley, 2007.

[3] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," 2016-. [Online]. Available: https://github.com/cowrie/cowrie

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513-523, 1988.

[5] T. Joachims, "Text categorization with support vector machines," in Proc. European Conf. Machine Learning (ECML), 1998, pp. 137-142.

[6] B. E. Strom et al., "MITRE ATT&CK: Design and philosophy," The MITRE Corp., Tech. Rep., 2020. [Online]. Available: https://attack.mitre.org

[7] J. Platt, "Probabilistic outputs for support vector machines," in Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61-74.

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[9] S. Ramirez, "FastAPI," 2018-. [Online]. Available: https://fastapi.tiangolo.com

[10] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," in Proc. 6th Int. Conf. Information Warfare and Security, 2011, pp. 113-125.